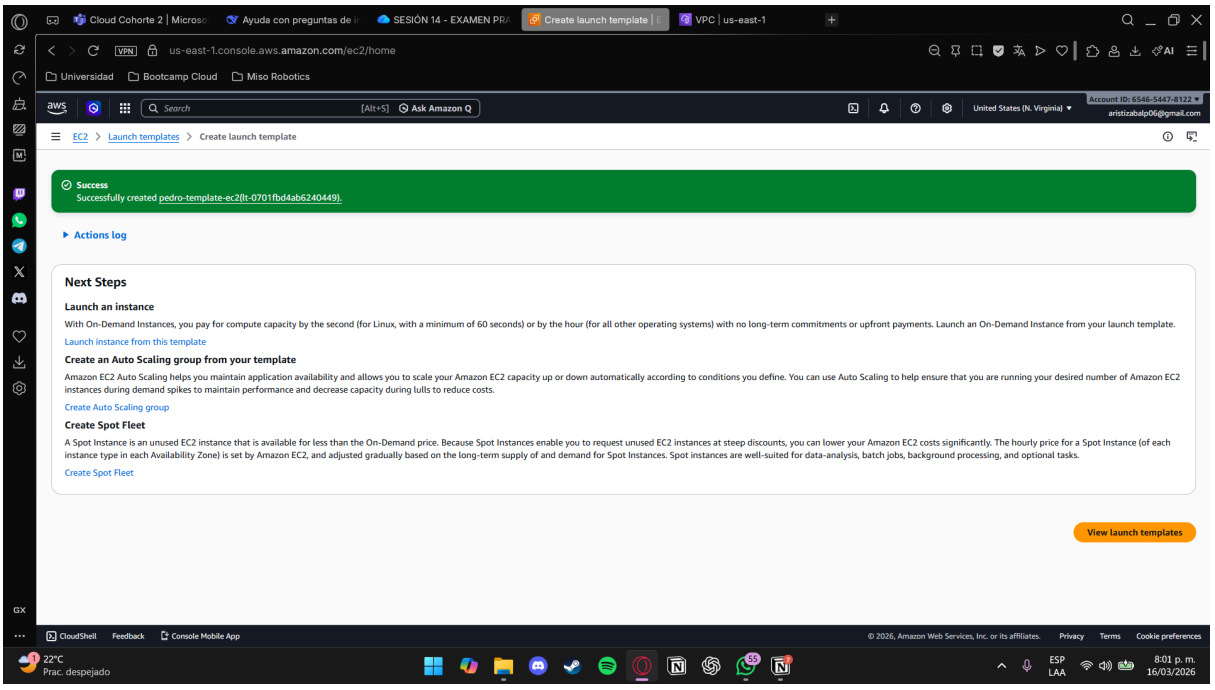
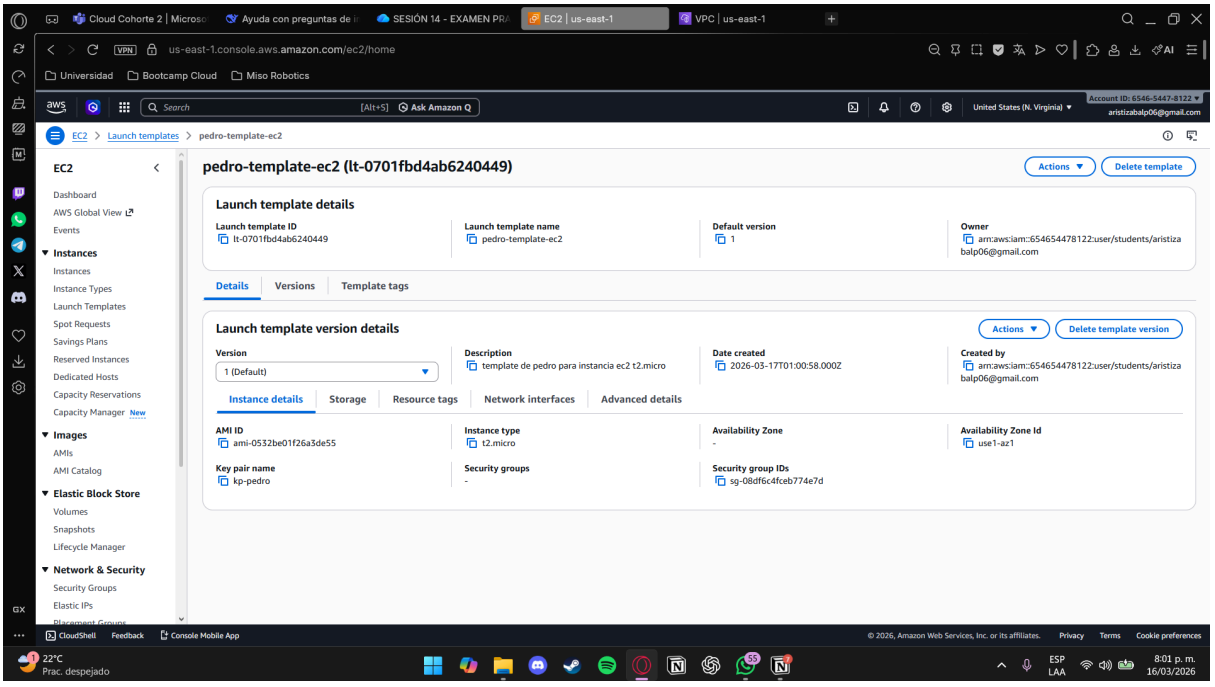
Examen 2 modulo 2

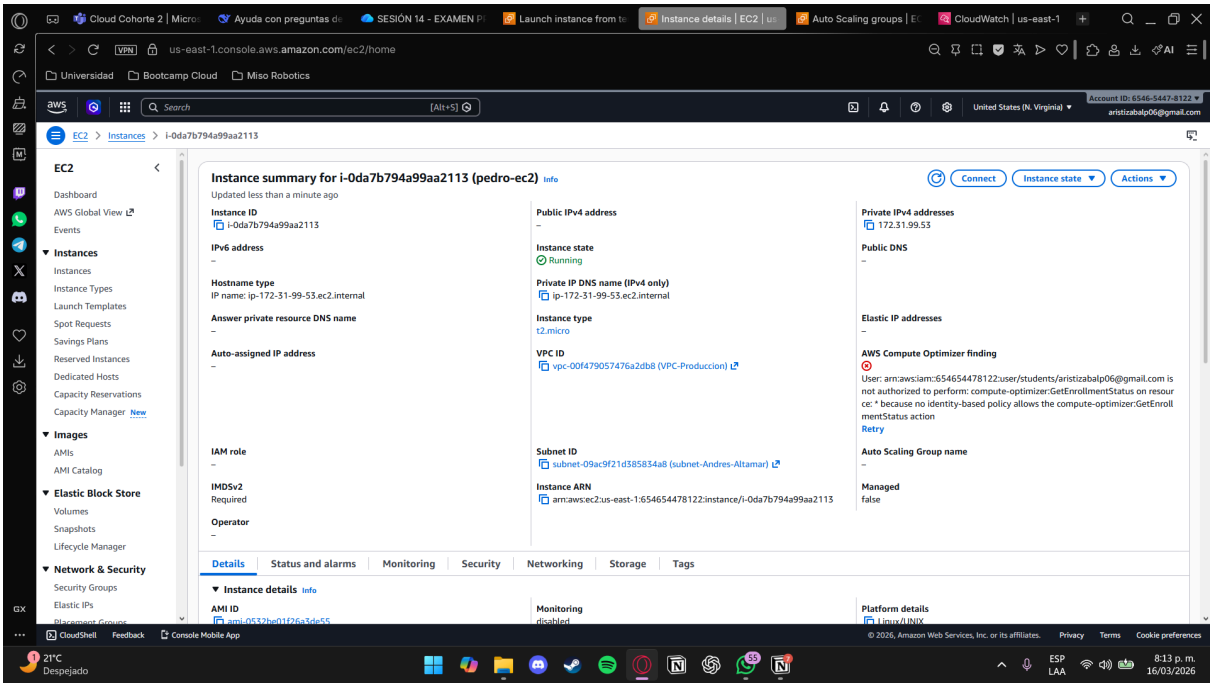
Parte 1

- Creamos el launch template

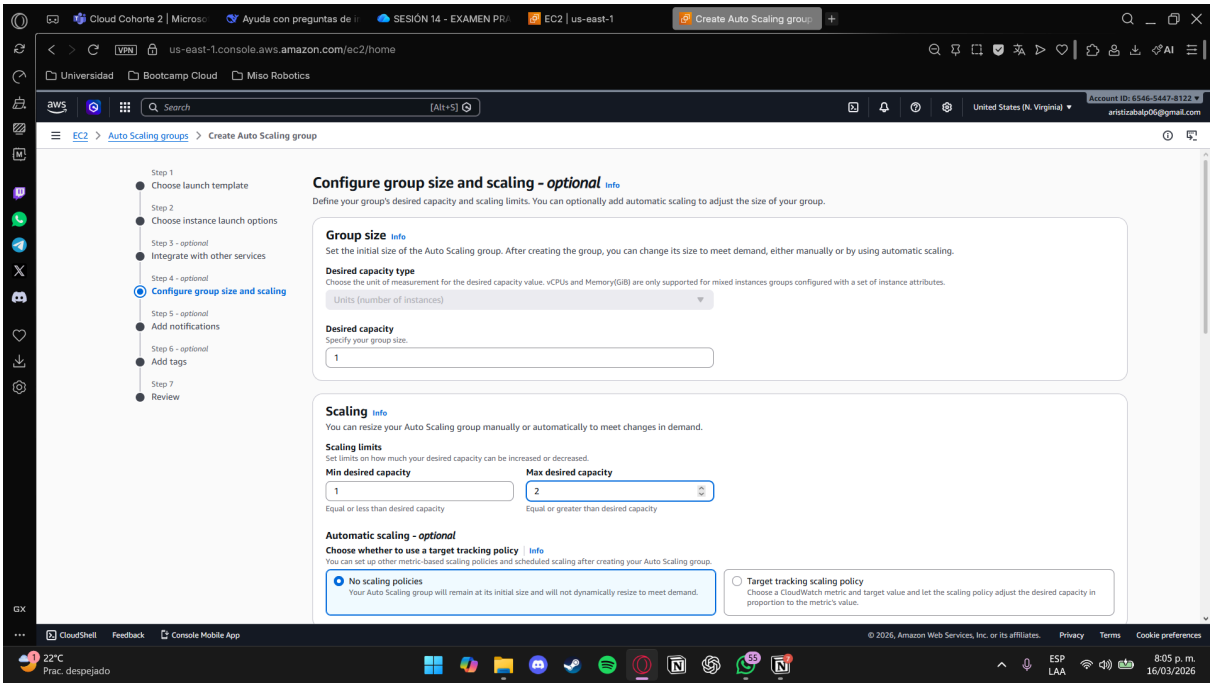


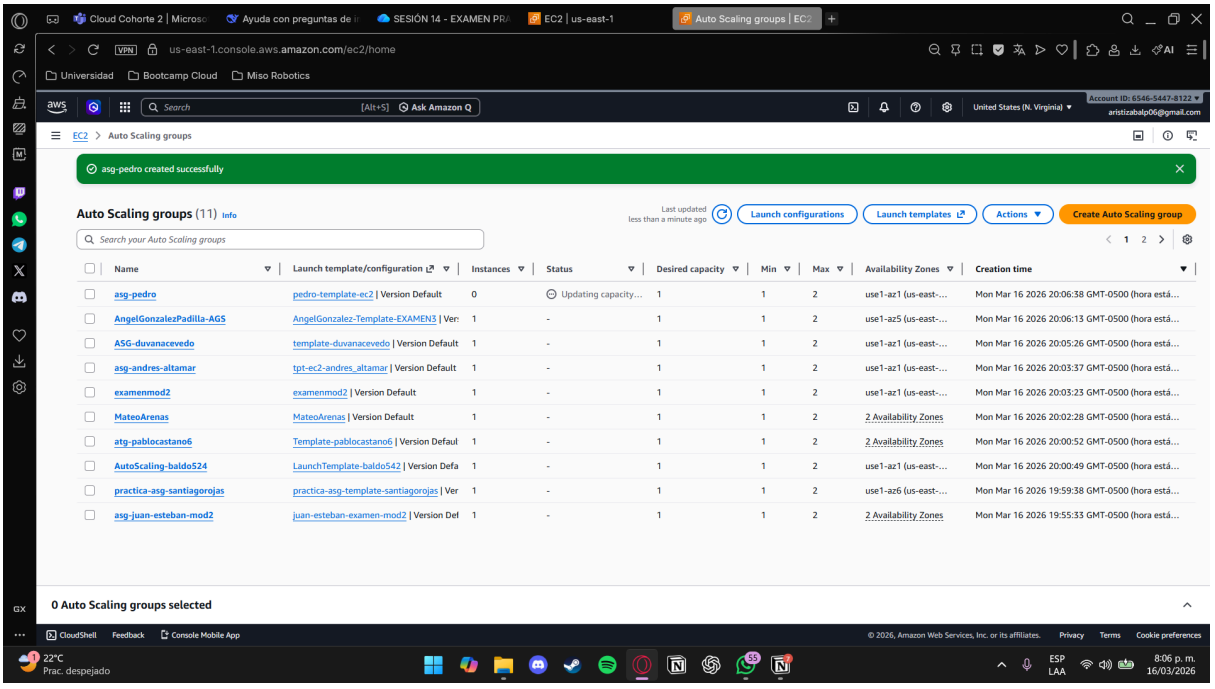


- Lanzamos la instancia EC2 a partir del template



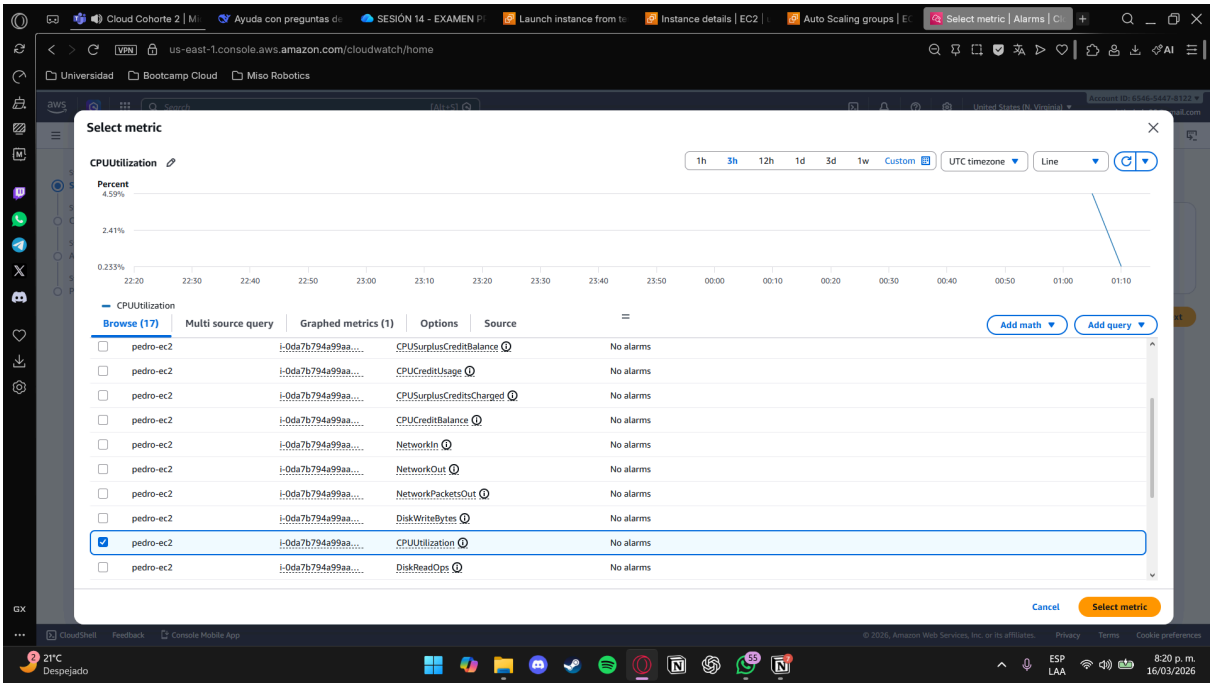
- Creamos el autoscaling group con los requerimientos pedidos



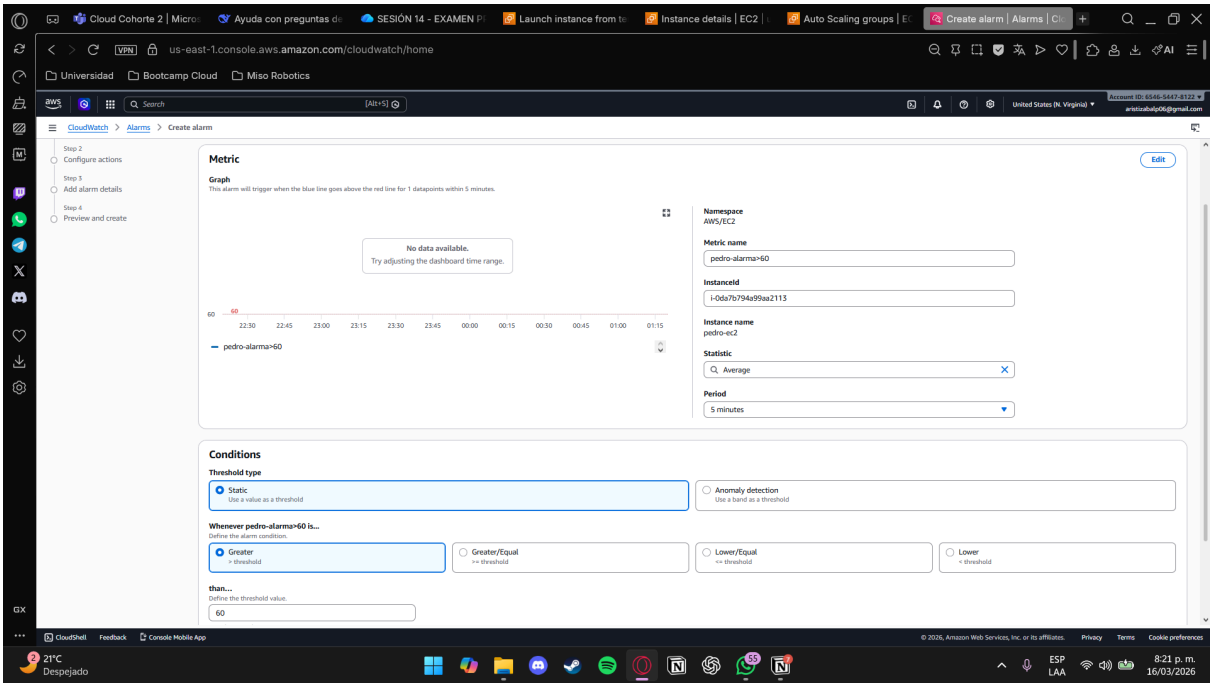


Parte 2

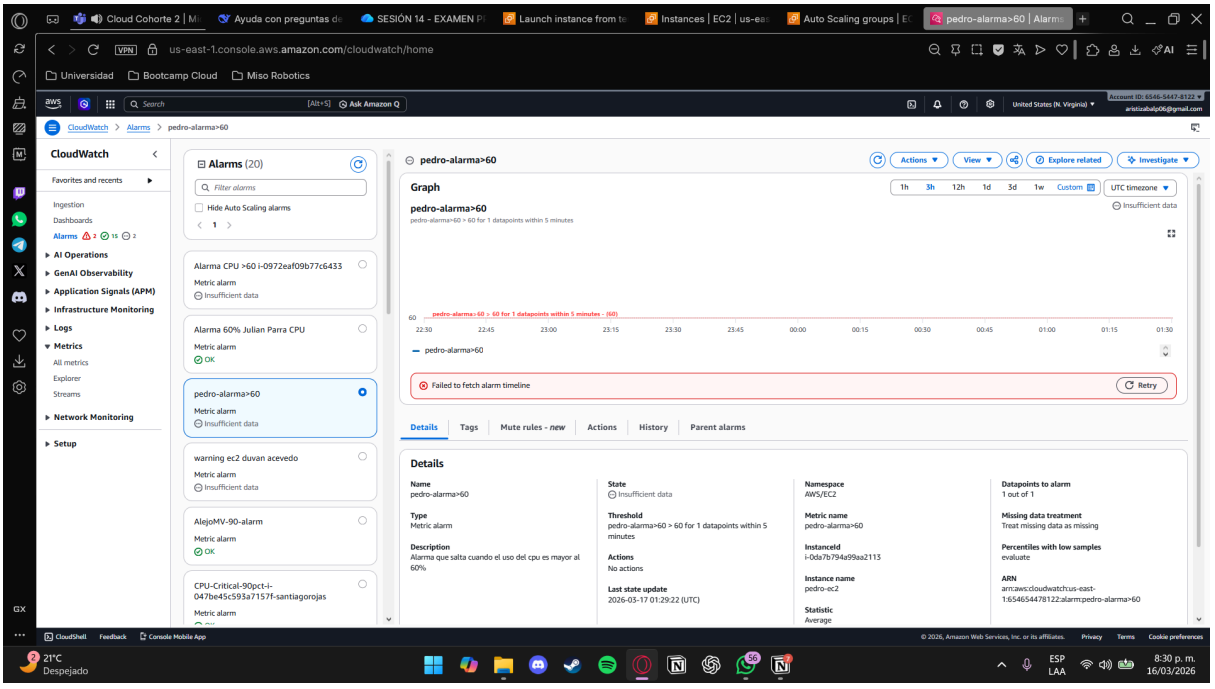
- Creamos la alarma en cloudwatch buscando por el ID de la instancia
- Seleccionamos CPU



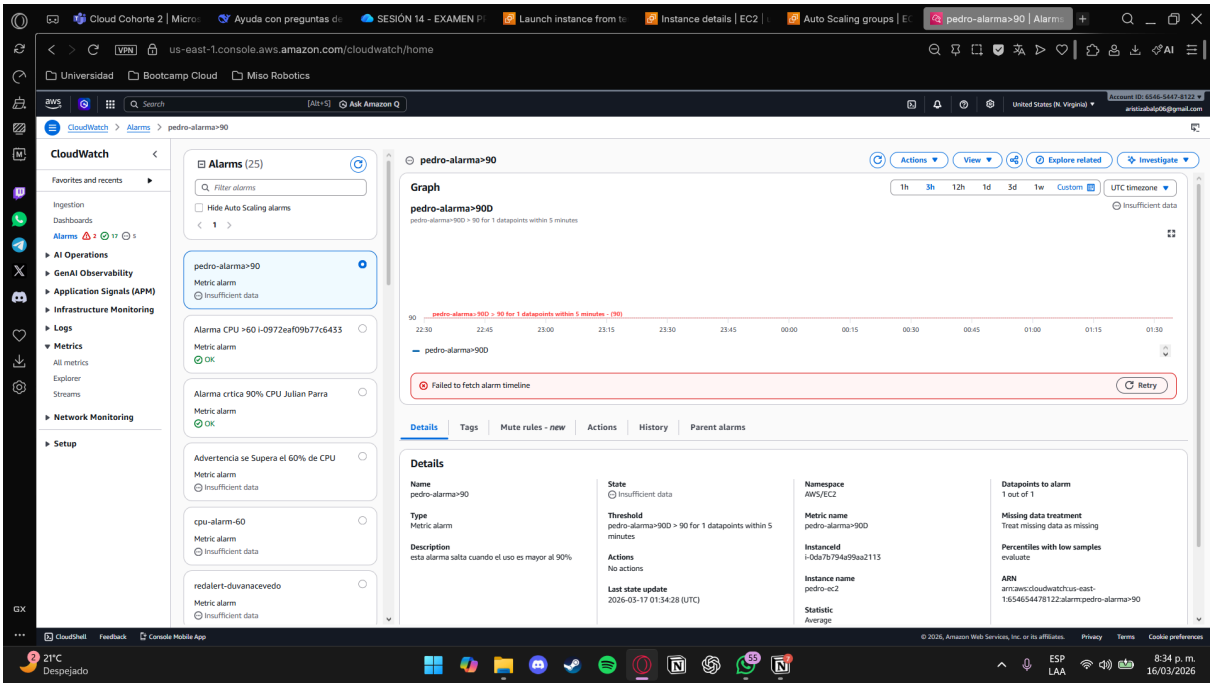
- Creamos primero la alarma para cuando el uso es mayor al 60%



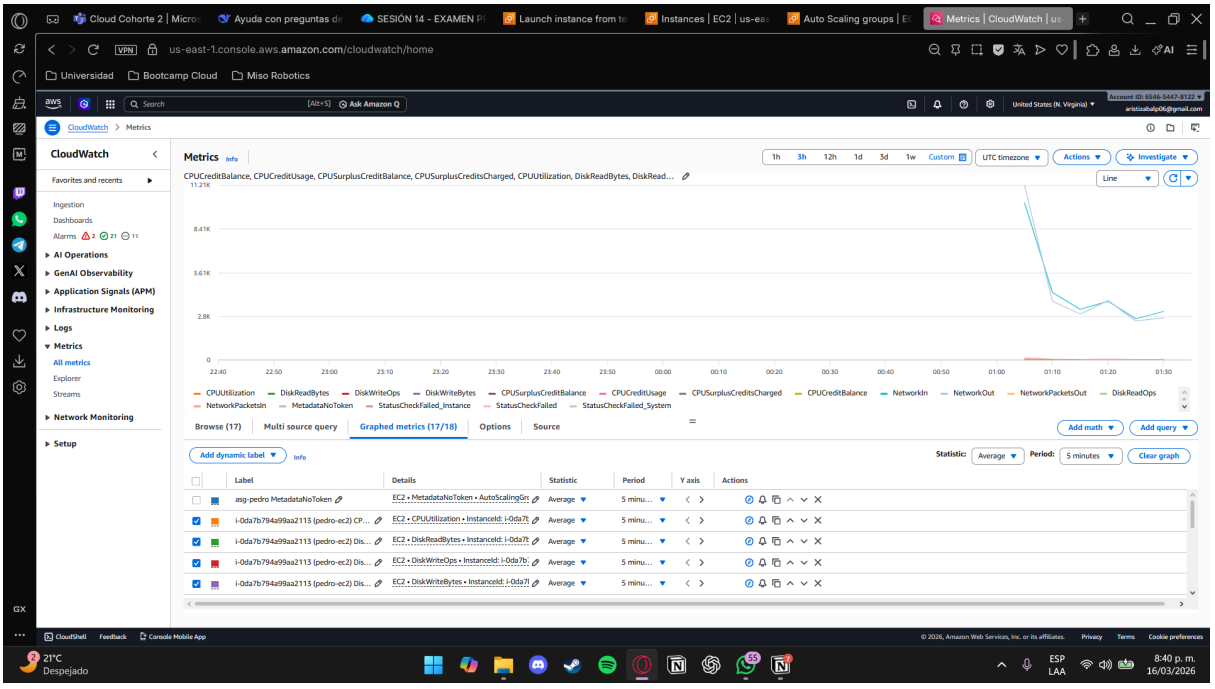
- Verificamos que la alarma fue creada (no salen datos porque lo deje en periodo de 5 minutos y tomé la captura recien creada la alarma)



- Creamos la segunda alarma para cuando el uso es mayor al 90% y verificamos que quedo creada

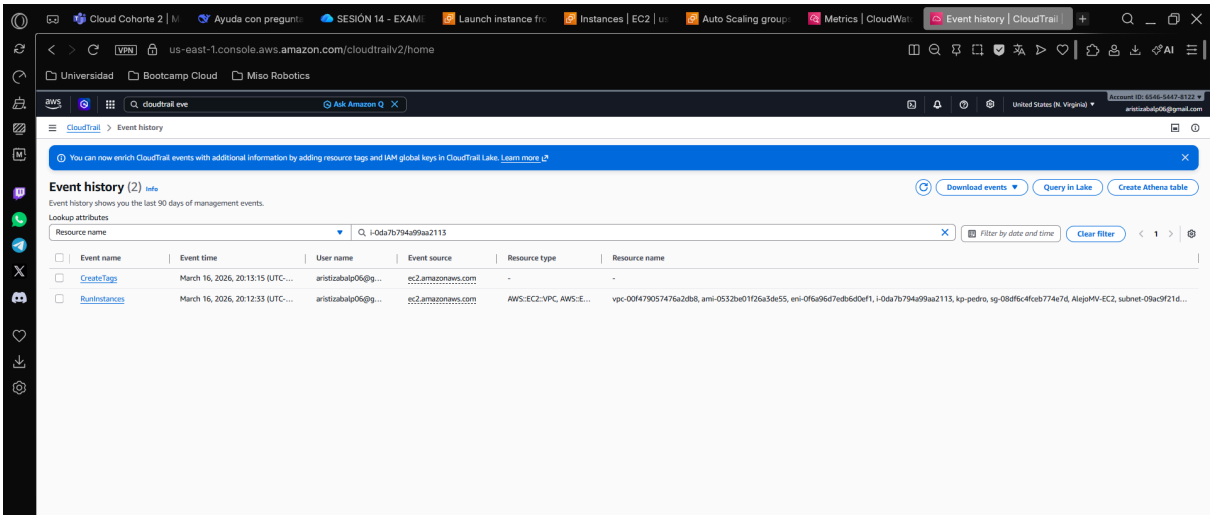


- Revisamos desde el cloudwatch las metricas asociadas al EC2



Parte 3

- Buscamos en cloudtrail por resource name y el ID de la instancia



- Ingresamos al evento RunInstances y verificamos el JSON donde se ve desde donde fue creado

{
"eventVersion": "1.11",
"userIdentity": {
"type": "IAMUser",
"principalId": "AIDAZQ3DSS4VLSFWTWKQJ",
"arn": "arn:aws:iam::654654478122:user/students/aristizabalp06@gmail.com",
"accountId": "654654478122",
"accessKeyId": "ASIAZQ3DSS4VJAC2GVNP",
"userName": "aristizabalp06@gmail.com",
"sessionContext": {
"attributes": {
"creationDate": "2026-03-17T00:46:05Z",
"mfaAuthenticated": "true"
}
}
},
"eventTime": "2026-03-17T01:12:33Z",
"eventSource": "ec2.amazonaws.com",
"eventName": "RunInstances",
"awsRegion": "us-east-1",
"sourceIPAddress": "191.95.147.165",
"userAgent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36 OPR/127.0.0.0",
"requestParameters": {

"instancesSet": {
"items": [
{
"imageId": "ami-0532be01f26a3de55",
"minCount": 1,
"maxCount": 1,
"keyName": "kp-pedro"
}
]
},
"instanceType": "t2.micro",
"blockDeviceMapping": {},
"availabilityZoneId": "use1-az1",
"monitoring": {
"enabled": false
},
"disableApiTermination": false,
"disableApiStop": false,
"clientToken": "d58844c5-3b36-4f1f-9b77-7a9db0fd6f58",
"networkInterfaceSet": {
"items": [
{
"deviceIndex": 0,
"subnetId": "subnet-09ac9f21d385834a8",
"groupSet": {
"items": [
{
"groupId": "sg-08df6c4fceb774e7d"
}
]
}
}
]
},
"launchTemplate": {
"launchTemplateId": "lt-0701fbd4ab6240449",
"version": "1"
}

},
"responseElements": {
"requestId": "23f286f5-825c-4222-862d-d04213196b35",
"reservationId": "r-0f0dc64f781314459",
"ownerId": "654654478122",
"groupSet": {},
"instancesSet": {
"items": [
{
"instanceId": "i-0da7b794a99aa2113",
"imageId": "ami-0532be01f26a3de55",
"bootMode": "uefi-preferred",
"currentInstanceBootMode": "legacy-bios",
"instanceState": {
"code": 0,
"name": "pending"
},
"privateDnsName": "ip-172-31-99-53.ec2.internal",
"keyName": "kp-pedro",
"operator": {
"managed": false
},
"amiLaunchIndex": 0,
"productCodes": {},
"instanceType": "t2.micro",
"launchTime": 1773709953000,
"placement": {
"availabilityZone": "us-east-1a",
"availabilityZoneId": "use1-az1",
"tenancy": "default"
},
"monitoring": {
"state": "disabled"
},
"subnetId": "subnet-09ac9f21d385834a8",
"vpcId": "vpc-00f479057476a2db8",
"privateIpAddress": "172.31.99.53",
"stateReason": {

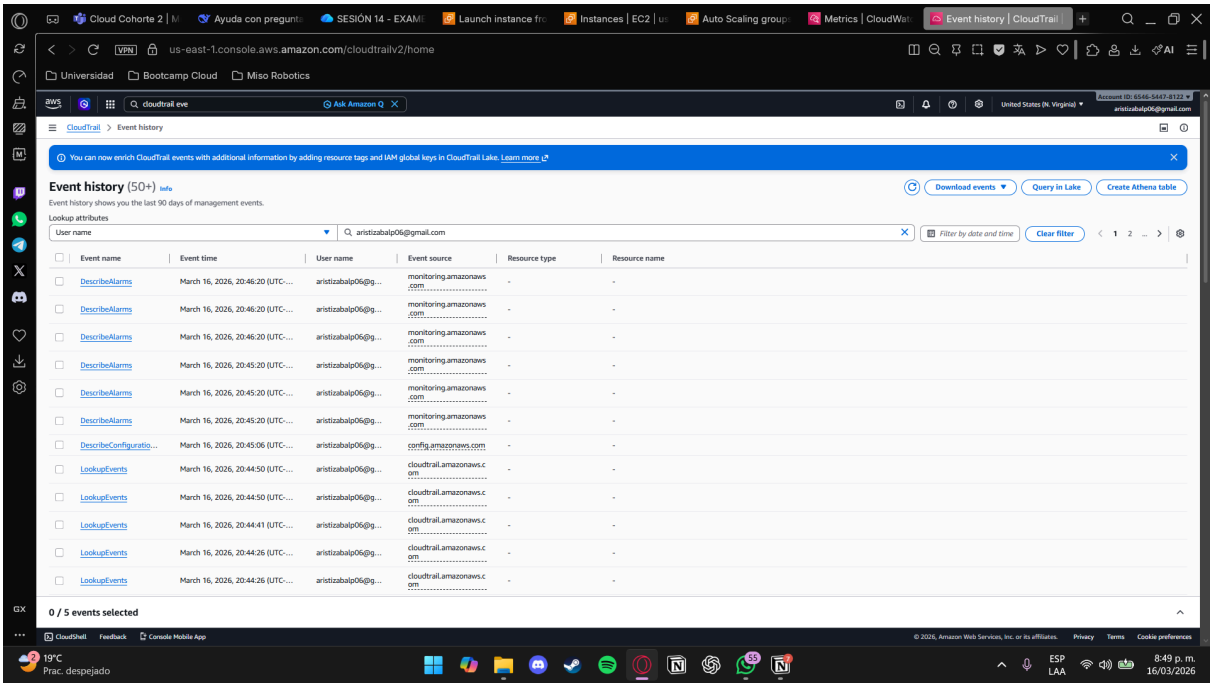
"code": "pending",
"message": "pending"
},
"architecture": "x86_64",
"rootDeviceType": "ebs",
"rootDeviceName": "/dev/xvda",
"blockDeviceMapping": {},
"virtualizationType": "hvm",
"hypervisor": "xen",
"tagSet": {
"items": [
{
"key": "aws:ec2launchtemplate:version",
"value": "1"
},
{
"key": "aws:ec2launchtemplate:id",
"value": "lt-0701fbd4ab6240449"
}
]
},
"clientToken": "d58844c5-3b36-4f1f-9b77-7a9db0fd6f58",
"groupSet": {
"items": [
{
"groupId": "sg-08df6c4fceb774e7d",
"groupName": "AlejoMV-EC2"
}
]
},
"sourceDestCheck": true,
"networkInterfaceSet": {
"items": [
{
"networkInterfaceId": "eni-0f6a96d7edb6d0ef1",
"subnetId": "subnet-09ac9f21d385834a8",
"vpcId": "vpc-00f479057476a2db8",
"ownerId": "654654478122",

"operator": {
"managed": false
},
"status": "in-use",
"macAddress": "02:7c:7a:12:d2:e7",
"privateIpAddress": "172.31.99.53",
"privateDnsName": "ip-172-31-99-53.ec2.internal",
"sourceDestCheck": true,
"interfaceType": "interface",
"groupSet": {
"items": [
{
"groupId": "sg-08df6c4fceb774e7d",
"groupName": "AlejoMV-EC2"
}
]
},
"attachment": {
"attachmentId": "eni-attach-04ef15c7cbc8a510c",
"deviceIndex": 0,
"networkCardIndex": 0,
"status": "attaching",
"attachTime": 1773709953000,
"deleteOnTermination": true
},
"privateIpAddressesSet": {
"item": [
{
"privateIpAddress": "172.31.99.53",
"privateDnsName": "ip-172-31-99-53.ec2.internal",
"primary": true
}
]
},
"ipv6AddressesSet": {},
"tagSet": {}
}
]

},
"ebsOptimized": false,
"enaSupport": true,
"cpuOptions": {
"coreCount": 1,
"threadsPerCore": 1
},
"capacityReservationSpecification": {
"capacityReservationPreference": "open"
},
"enclaveOptions": {
"enabled": false
},
"metadataOptions": {
"state": "pending",
"httpTokens": "required",
"httpPutResponseHopLimit": 2,
"httpEndpoint": "enabled",
"httpProtocolIpv4": "enabled",
"httpProtocolIpv6": "disabled",
"instanceMetadataTags": "disabled"
},
"maintenanceOptions": {
"autoRecovery": "default",
"rebootMigration": "default"
},
"privateDnsNameOptions": {
"hostnameType": "ip-name",
"enableResourceNameDnsARecord": false,
"enableResourceNameDnsAAAARecord": false
}
}
]
}
},
"requestID": "23f286f5-825c-4222-862d-d04213196b35",
"eventID": "d2aa4abd-8a98-48d9-82d9-0b8270943c1e",
"readOnly": false,

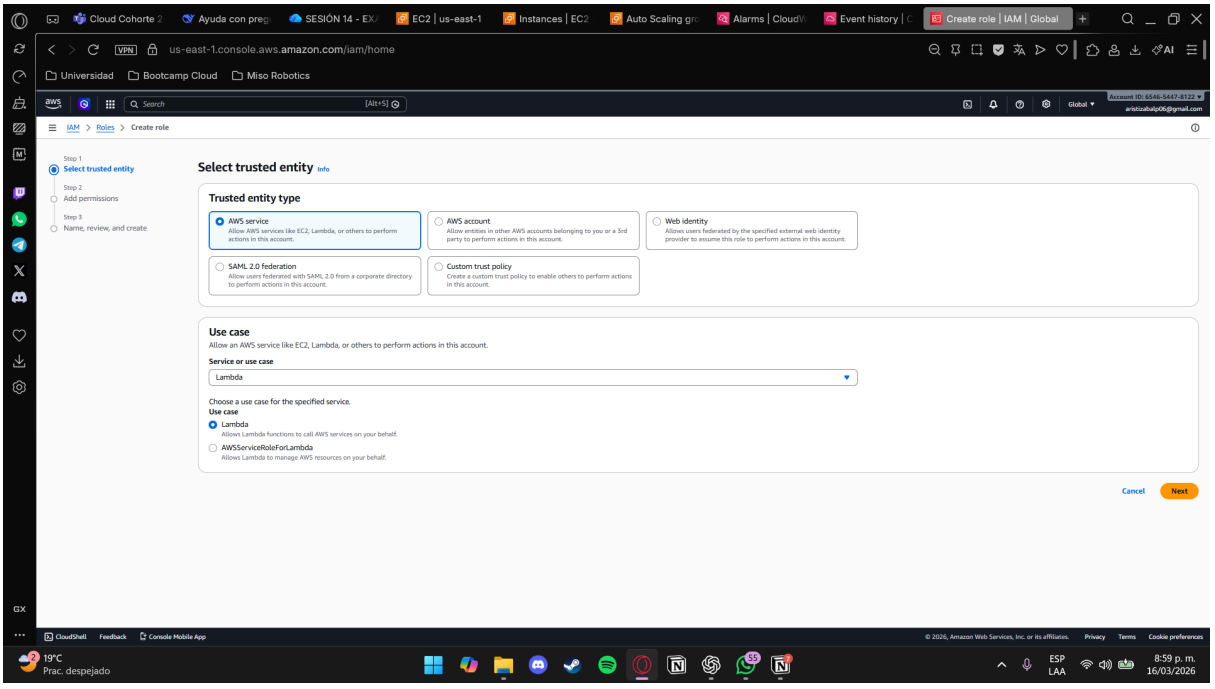
"resources": [
{
"accountId": "654654478122",
"type": "AWS::EC2::Instance",
"ARN": "arn:aws:ec2:us-east-1:654654478122:instance/i-0da7b794a99aa2113"
}
],
"eventType": "AwsApiCall",
"managementEvent": true,
"recipientAccountId": "654654478122",
"eventCategory": "Management",
"tlsDetails": {
"tlsVersion": "TLSv1.3",
"cipherSuite": "TLS_AES_128_GCM_SHA256",
"clientProvidedHostHeader": "ec2.us-east-1.amazonaws.com"
},
"sessionCredentialFromConsole": "true"
}

- Buscamos por user de IAM y listamos las ultimas acciones

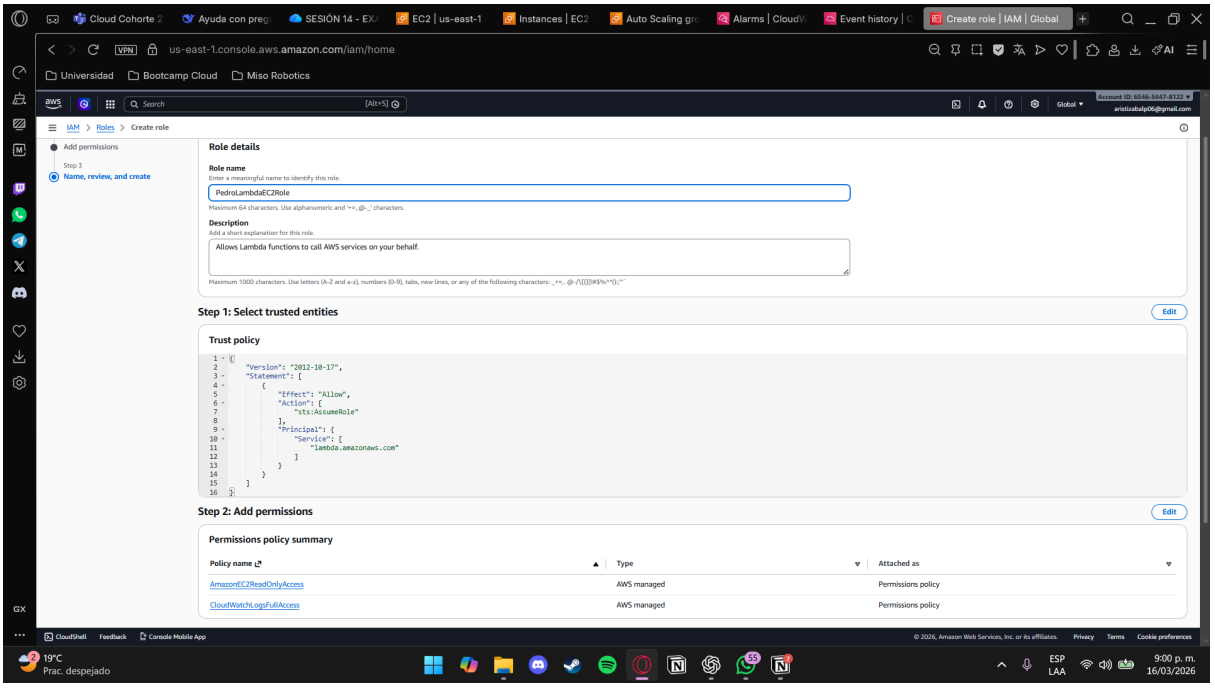


Parte 4

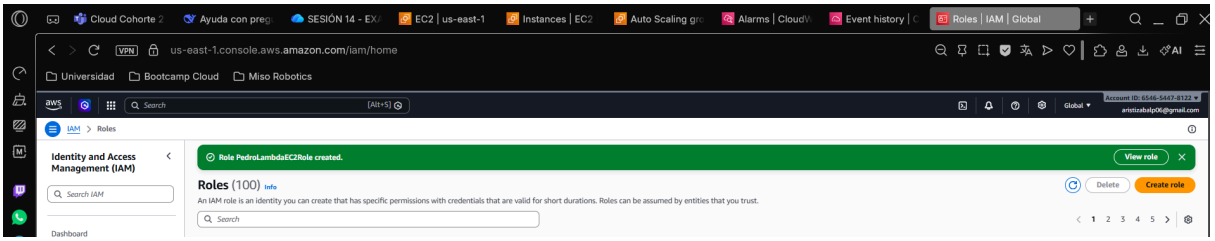
- Creamos el rol para la lambda



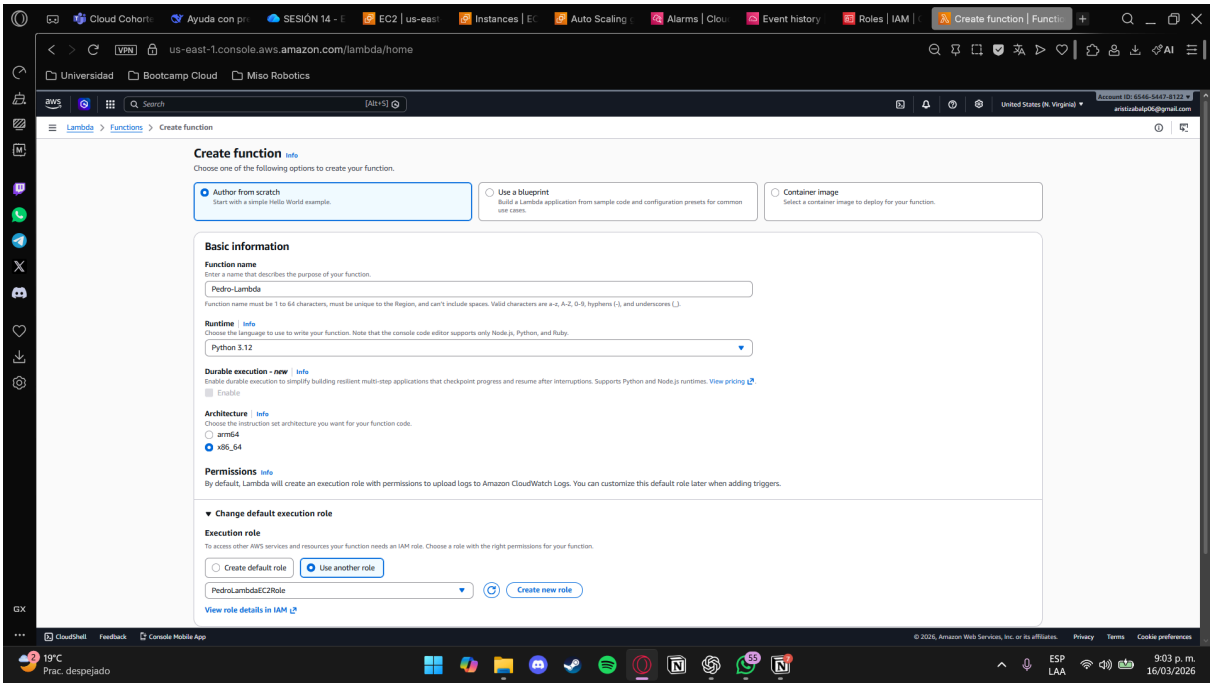
- Seleccionamos las politicas:
- **AmazonEC2ReadOnlyAccess**
- **CloudWatchLogsFullAccess**



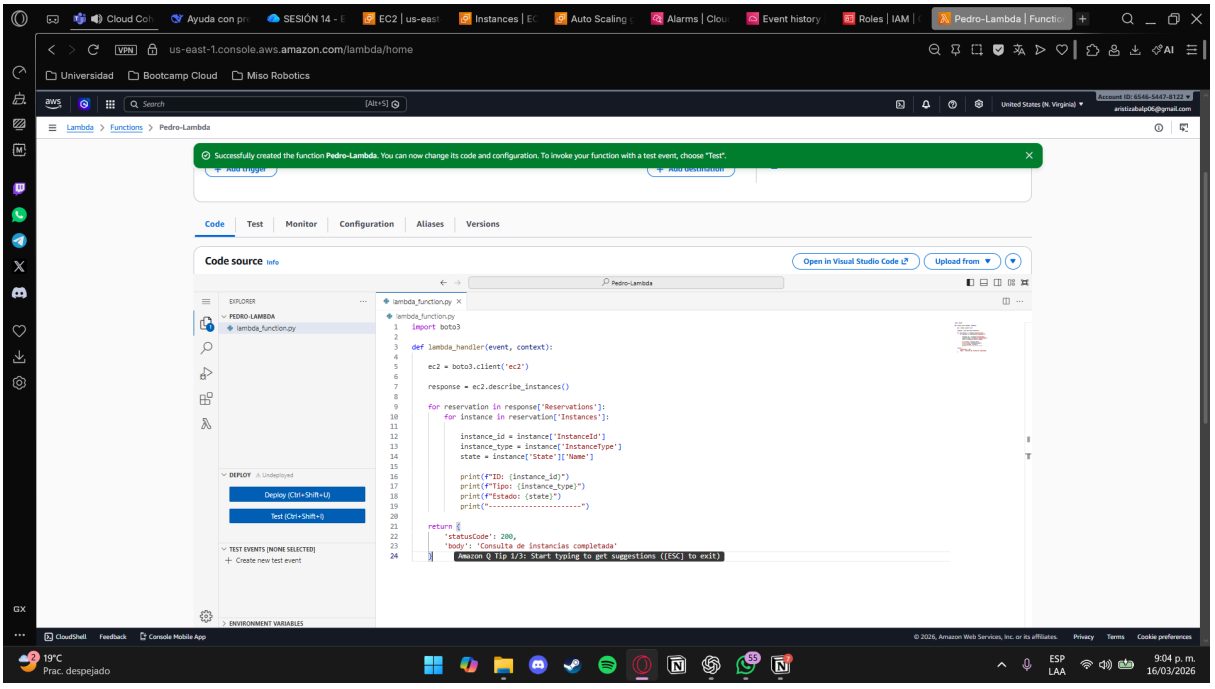
- Verificamos que el rol quedo creado



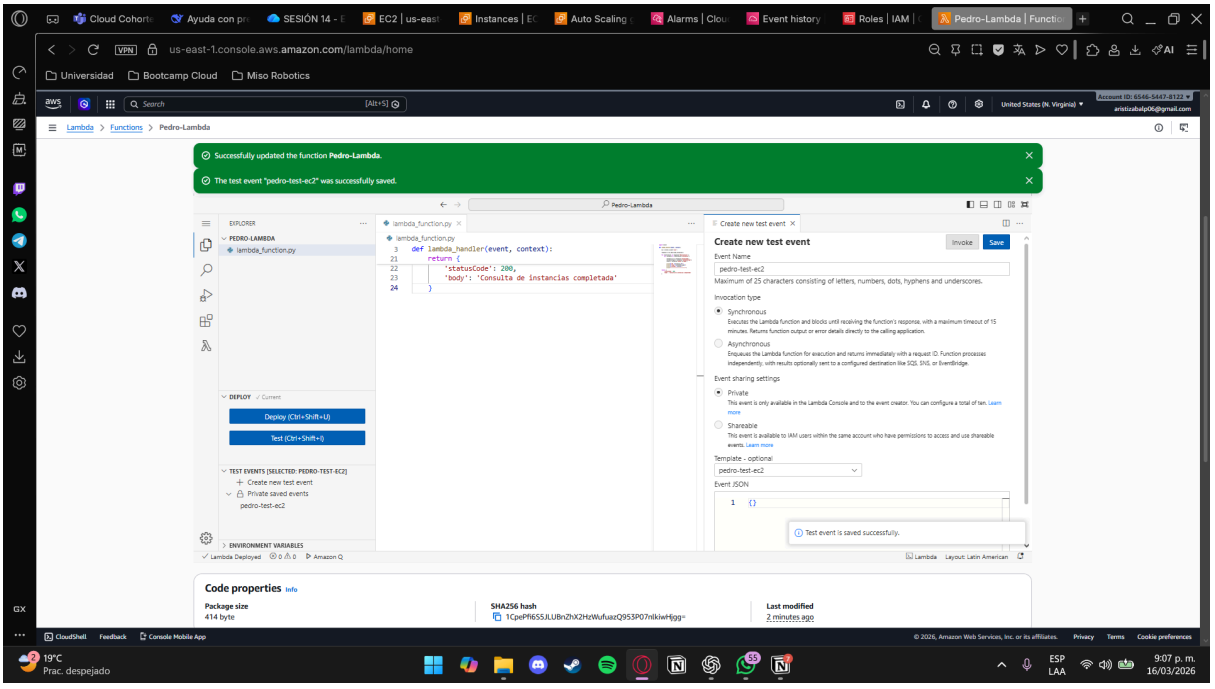
- Creamos la funcion de la Lambda



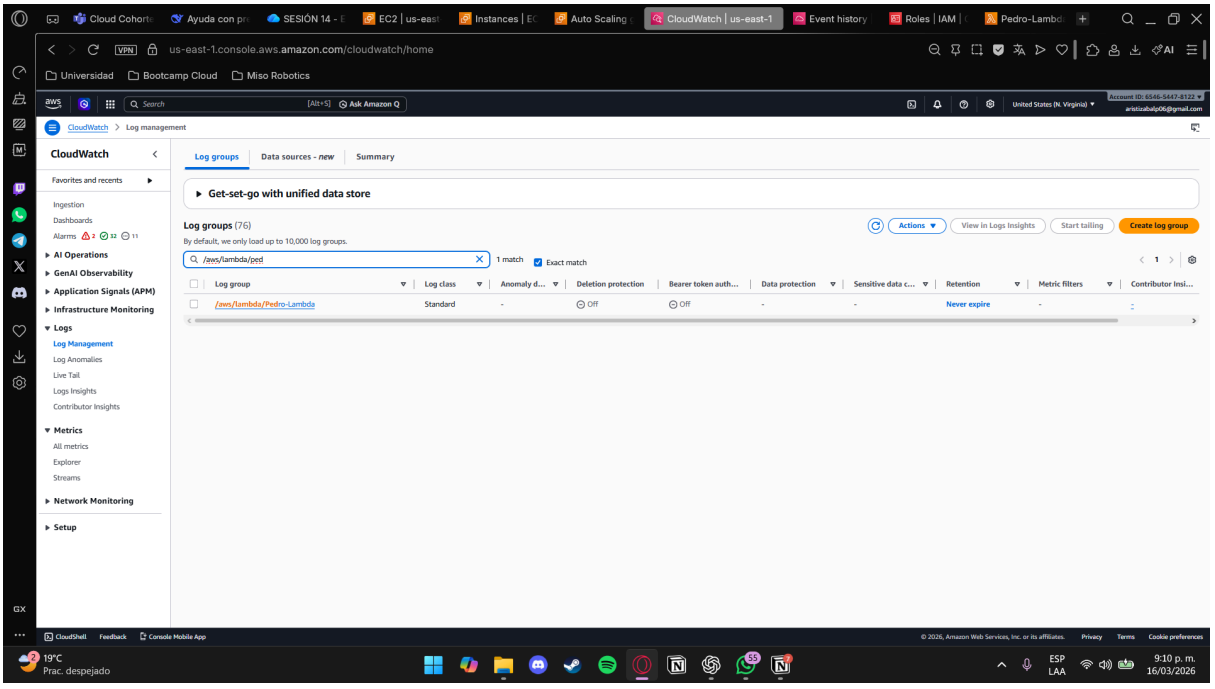
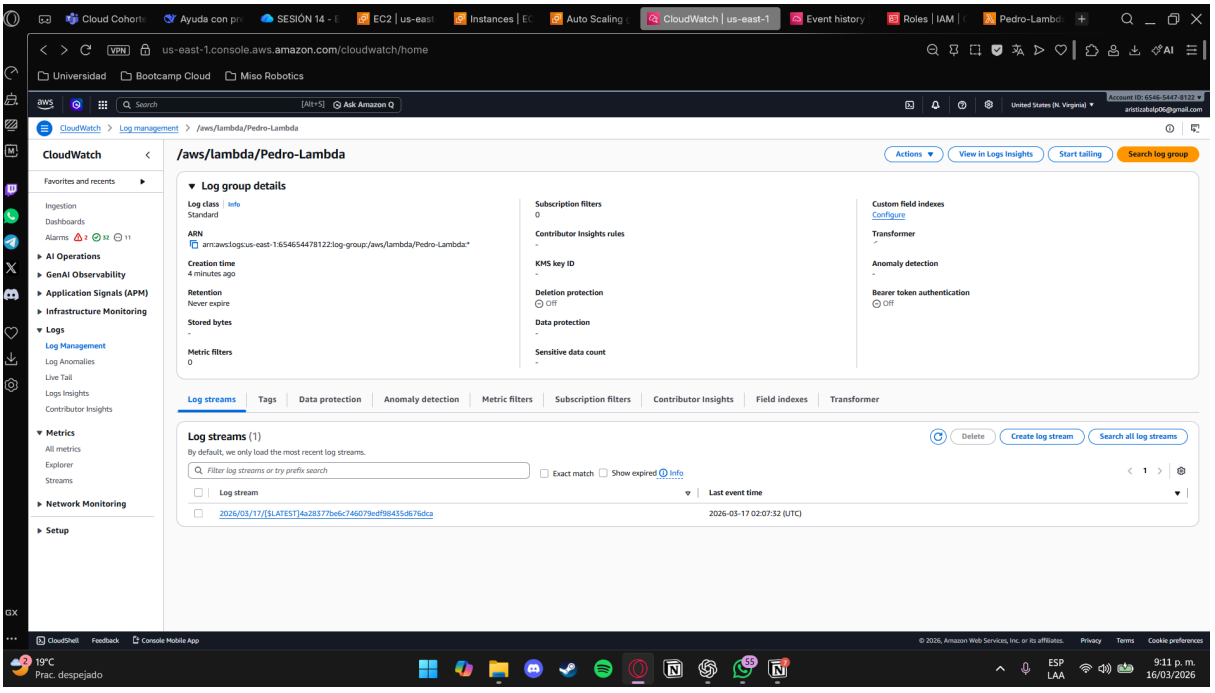
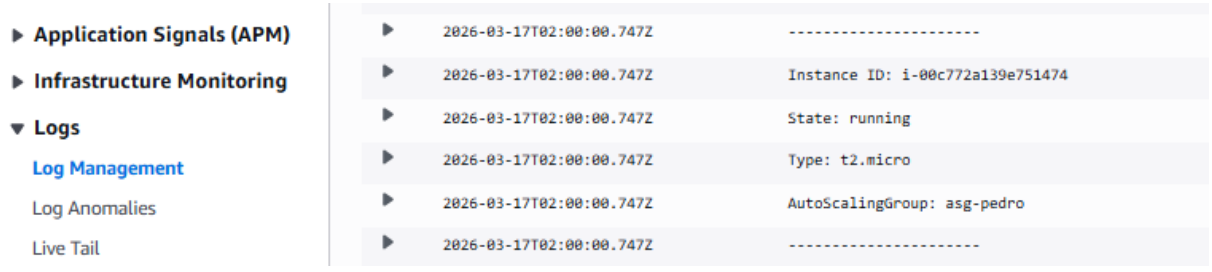
- Verificamos que la lambda fue creada y usamos un codigo proporcionado por ChatGPT, y le damos en deploy



- Creamos un test



- Vamos a cloudwatch → Log management → filtramos por el grupo de mi lambda



- Verificamos que efectivamente esta el Log de la lambda que implementamos previamente